Episodio 11 de 14: El veredicto y los límites

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** episodio 10, «Sin Docker: el error escondido y la caché que se llena».
- **Episodio siguiente:** episodio 12, «Juicio a la réplica (1): escala, método y Folly».

Lo esencial de este episodio:

- Veredicto: de 15 afirmaciones, 9 se reproducen, 2 en parte, 3 no y 1 no se confirma.
- No se reproducen: que Autores gane a Folly, que gane a Anderson con muchas pilas y que su store sea el segundo con un hilo.
- Los límites: sin 4 sockets ni NUMA, recortes con sesgo, latencia ruidosa con contención, un solo módulo de RAM y preguntas abiertas como Folly o Williams entre clústeres.

Parte 7. El veredicto, afirmación por afirmación

Cada fila es una frase del paper puesta a prueba. Las cifras son de la tanda con Docker; entre paréntesis, lo que dio la tanda sin Docker con los mismos programas oficiales.

#	Lo que dice el paper	Resultado aquí	Veredicto
1	Autores es mucho más rápido que Libstdc++	Gana con contención (0,06× a 0,16×; sin Docker, 0,05× a 0,17×) y empata con n = 10^6	**Se reproduce**
2	Autores es mucho más rápido que Williams	Gana en los 6 escenarios (0,24× a 0,73×; sin Docker, 0,25× a 0,76×)	**Se reproduce**
3	Autores es mucho más rápido que Folly	Gana en 3 escenarios, empata en 2 y pierde en 1, con y sin Docker	**No se reproduce**; la brecha ya existe con 1 hilo, y no se debe ni a Docker ni al recorte
4	Con poca contención por u pequeño, Autores supera a Anderson	Gana con u = 1 %: un 6 % con n = 1 y un 65 % con n = 10 (sin Docker, un 10 % y un 75 %)	**Se reproduce**
5	Con poca contención por n grande, Autores supera a Anderson	Con n = 10^6 pierde hasta 18 hilos y empata a partir de 24 (sin Docker, lo mismo)	**No se reproduce** en este PC
6	Con mucha contención, las actualizaciones de Autores son más lentas y Anderson gana	Anderson 1,87× a 1,99× con n = 1 y u ≥ 10 % (sin Docker, 1,90× y 1,93×)	**Se reproduce**

#	Lo que dice el paper	Resultado aquí	Veredicto
7	El `load` de Autores es siempre el de menor latencia	El más bajo en los tres casos, con y sin Docker. Con contención, por poco: con 15 rondas, sus rangos se solapan con los de Anderson y Folly	**Se reproduce**, con poco margen
8	El `store` de Autores es el segundo con p = 1, tras Anderson	Es el cuarto de cinco, por 5 ns, con y sin Docker	**No se reproduce**
9	El `store` de Autores es el mejor con contención	Empate con Anderson, con y sin Docker	**En parte**
10	`store & reclaim` es más lento que `store` con p = 1	Los cinco, con y sin Docker	**Se reproduce**
11	Con contención, algunos van más rápido en `store & reclaim`	Con Docker y 5 rondas, Autores y Libstdc++. Sin Docker y con 15 rondas, la mejora se queda en un 1-4 %, dentro del ruido	**No se confirma**; la causa propuesta tampoco se comprobó
12	La latencia de Williams es casi siempre mayor	Sin contención es la mayor en las 3 operaciones. Con contención supera a Libstdc++ en los 6 casos y a Folly en 3 (sin Docker, en 5 y en 2)	**En parte**
13	Anderson tiene un pequeño sobrecoste de memoria; Autores, ninguno	Anderson hasta 19 960 bloques (469 KB); los demás, bajo el límite (sin Docker, 20 256 y 476 KB)	**Se reproduce**
14	La recarga reintenta como mucho P veces	Máximo 8 con P = 32, en millones de recargas	**Se reproduce**
15	No hay use-after-free	0 errores con ASan y TSan en Autores	**Se reproduce** en lo probado

Las filas de latencia, con más rondas. Las filas 7, 9, 11 y 12 se apoyaban en puntos con p = 24 y p = 56, donde una misma medida cambia varias veces de una ronda a otra. Por eso se repitieron sin Docker con 15 rondas en vez de 5 (Parte 6.2). Resultado: la 7 y la 9 se sostienen como estaban, la 12 se sostiene algo más débil y la 11 deja de confirmarse. Aun así, cualquier diferencia pequeña con contención hay que leerla como empate.

Dicho en una frase: la idea central del paper sale bien parada. El algoritmo es correcto, cumple su cota, no acumula basura y lee rápido. Lo que no se sostiene aquí es la superioridad frente a Folly ni la ventaja con muchísimas pilas, y tampoco se pudo reproducir la escala de 4 sockets.

¿Y sin Docker? Con los mismos programas oficiales, fuera de Docker, la tabla sale igual salvo la fila 11, que deja de confirmarse con 15 rondas. Las filas que dependían de Folly y de Anderson (3, 4, 5, 6 y 13) ya se han podido contrastar, y dan lo mismo que con Docker. El detalle fila por fila está al final de la Parte 6.6.

Parte 8. Limitaciones, dicho sin rodeos

Sobre la máquina

- **No hay 4 sockets.** La simulación usa clústeres de núcleos E. El castigo por distancia es de 1,35×, frente a unas 3× en máquinas parecidas a la del paper, y ese 3× es un dato indirecto, porque no viene de la máquina del paper.
- **No hay NUMA.** Toda la RAM está igual de lejos. `numactl -i all` aquí no hace nada.
- **No hay SMT**, y son 24 núcleos, no 72. La zona de más de 54 hilos del paper queda fuera de alcance.
- **Un solo módulo de RAM.** 32 GB de DDR5-5600 y la otra ranura vacía: un solo canal. Eso limita el escenario n = 10^6, y el paper tenía mucha más memoria en paralelo.
- **La CPU es híbrida.** En `sistema`, Linux reparte los hilos entre núcleos P y E, de potencia distinta. Con 1 hilo suele usar un núcleo P. El paper no tenía ese problema.
- **Frecuencia sin fijar.** Se dejó el regulador como estaba. Llegaron a 90 °C o más 5 de las 3 720 ejecuciones con Docker y 26 de las 9 796 sin Docker; el máximo fue 99 °C. Están repartidas y no se descartaron.

Sobre el método

- **Recortes:** 2 s × 5 en vez de 10 s × 7, y 3 rondas en afinidad. La validación de la Parte 6.7 muestra desviaciones de hasta un 21 % en los 2 escenarios validados con Docker, y de hasta un 19 % en los 6 validados sin Docker. Además, sin Docker se vio que el recorte tiene un sesgo: da más con contención y menos con n = 10^6. No se sabe por qué.
- **Valores del paper:** los de la figura 5 salen de la posición de los marcadores y los de la figura 6, de las etiquetas impresas. Son muy precisos, pero no son sus datos crudos.
- **Williams en latencia** se midió después que los demás, con un binario aparte. Al activar su línea se recompiló el binario de los otros cuatro, así que se comparó el binario nuevo con el anterior en 8 puntos: 7 coincidieron, con diferencias de un 7 % como mucho, y el octavo era el punto bimodal, con los dos modos presentes en ambos binarios.
- **Dos intentos de latencia de Williams se descartaron, los dos por errores nuestros.** El primero falló (54 ejecuciones) por un error de `baterias.py`. El segundo salió sin fallos, pero midió un Williams con candado (162 ejecuciones), por compilarlo sin `-mcx16`. Las medidas válidas son las del tercer intento.
- **Interferencias:** el PC es de escritorio y no quedó aislado. Durante la tanda sin Docker se ejecutó varias veces `analizar.py`, a prioridad mínima y unos 3 segundos cada vez. Durante la prueba de pares y los controles también se hicieron lecturas y compilaciones breves, a prioridad mínima. El orden aleatorio reparte ese efecto.
- **La medición de RAM casi nos engaña.** La primera versión miraba los últimos segundos de cada ejecución, y ahí el programa ya estaba liberando memoria: dio 14 GB/s para Anderson y apuntaba a que la RAM no era el freno. Lo delató que el tráfico no crecía con el rendimiento. La versión corregida solo mira la mitad central de ejecuciones más largas y da 34 GB/s, justo lo contrario. Además, el contador avanza a saltos de 1 GB, y con 1 hilo las dos rondas difieren hasta un 41 %, así que sus valores de

1 hilo son orientativos. Con 12 hilos o más difieren un 6 % como mucho. Solo hay 2 rondas.
- **`bench_basura`** es una prueba propia; su «límite» es una fórmula nuestra, no del paper.
- **ASan y TSan** se pasaron a los seis algoritmos, salvo TSan en Folly y Anderson, y durante pocos segundos por prueba (Parte 6.5).

Sobre las conclusiones

- Quedan **preguntas sin responder**: por qué Folly va tan distinto, por qué Autores va por detrás con n = 10^6 y pocos hilos, por qué algunos ganan al separarse, y por qué Williams se hunde entre clústeres en el programa oficial (y más dentro de Docker que fuera). Las explicaciones que damos para esos casos son hipótesis y están marcadas como tales. Ninguna de ellas cambió al medir sin Docker.
- **El techo de n = 10^6** ya se midió, y apoya la explicación de la memoria, pero no la demuestra (Parte 6.1). Haría falta repetirlo con dos módulos de RAM.

Sobre la versión sin Docker

- **Ya está completa, pero costó.** Los binarios oficiales necesitan compilar Folly y ParlayLib fuera del contenedor, con `preparar.sh`. El sistema de permisos de la sesión del asistente bloqueó esa orden («Code from External»: compilar y ejecutar en el sistema código descargado), y el bloqueo no se saltó por otra vía. Lanzada a mano, compiló Folly y ParlayLib y se paró al configurar `bench_stack`. La causa resultó ser `fmt`: su archivo de CMake sube 4 carpetas para hallar su raíz, pero el enlace de `preparar.sh` estaba solo a 3 carpetas de `install/fmt`, así que buscaba `install/lib/...` y no existía. La corrección añade dos enlaces, y **funcionó**: con ella, `pendientes.sh` compiló los binarios oficiales y los midió la noche del 6 al 7 de noviembre de 2026, sin un solo fallo.
- **Que Docker no influya ya está medido**, con los programas oficiales: 259 de 270 puntos cambian menos de un 10 % (Parte 6.6). Pero dentro y fuera cambian también el compilador, glibc, Boost, jemalloc y la forma de compilar Folly. Así que una diferencia concreta, como la de Williams entre clústeres, no se puede achacar a Docker sin más.
- **Las tandas con `bench.cpp` son otra prueba.** Salen de otro programa, y la Parte 6.6 detalla qué cambia. Sus paneles n = 10 sin alinear están frenados por la compartición falsa.
- **La tanda alineada no se midió alternando** con la de sin alinear: se hizo horas después. El Mutex, que casi no cambia, hace de control, pero no es lo mismo que alternar. La prueba de la RAM de después sí alternó con n = 10^5 y n = 10^6, y con n = 10^6 dio los mismos cocientes. Con n = 1 y n = 10 no se repitió alternando. Y su latencia no se midió: `nativo_alineado/latencia.csv` solo tiene la cabecera.
- **Temperatura:** llegaron a 90 °C o más 8 de las 2 976 ejecuciones con `bench.cpp` (máximo 99 °C), 17 de las 4 660 con los programas oficiales (máximo 98 °C) y 1 de las 2 160 con la copia alineada (máximo 93 °C).

Cosas que el equipo debería revisar. No se han tocado sus archivos; esto queda a su criterio:

Dónde	Qué pasa
`replica/informe/02_latencia.md`, línea 18	Pone libstdc++ `load` p = 56 = 86 700 ns. La figura 6 del paper pone **99,7 µs**

Dónde	Qué pasa
`README.md`	Algunos números no coinciden con `capturas/rendimiento.csv` (por ejemplo, 34,3 frente a 39,126)
`codigo/ejecutar_pruebas.sh`	Su paso de TSan falla en este núcleo con `FATAL: unexpected memory mapping`: le falta `setarch -R`, que `correr_tsan.sh` sí tiene
`codigo/bench.cpp`, línea 73	`Pila` no está alineada: con n = 10 hay compartición falsa, y Autores rinde hasta 3,65 veces menos. Con `template <class I> struct alignas(128) Pila` sube de 32 a 119 Mop/s (probado en una copia). Afecta a sus paneles n = 10
`codigo/bench.cpp`	Frente a la receta oficial usa pilas de 16/2 en lugar de 20 + T/n, no usa jemalloc ni numactl, y n = 10^5 en lugar de 10^6
`codigo/williams/`	Usa `std::atomic`, y el artefacto, `boost::atomic`. No son exactamente el mismo Williams: `boost::atomic` rindió entre un 12 % y un 25 % más con 24 hilos
CMake de latencia del artefacto	Compila sin `-mcx16`. Quien active a Williams ahí lo mide con un candado escondido